Intelligent Routing Protocol

Computer Networks Project Report

Slot F2 · Team 18

Neha Damani · 24BCE2431

Thipireddy Bhuvana Kruthi · 24BCE2430

Afreen Faizal · 24BCE2514

Source code: https://github.com/nehadamani0909/F2_Team18

Contents

1. The project in one minute 3
2. Background terms you need 3
3. The problem it solves 4
4. How the protocol works 4
5. The cost formula, explained 5
5.1 Each part in plain words 5
5.2 Why −ln(1 − p) for loss? 5
5.3 Worked example (real output from the demo) 6
6. Keeping it stable 6
7. The code, file by file 6
8. How it was tested 7
8.1 The test network 7
8.2 The nine experiments 7
8.3 What it is compared against 8
8.4 What is measured 8
9. Results 8
9.1 What the results mean 9
10. How to run it 10
11. How to pitch it (about 5 minutes) 10
12. Questions your teacher may ask 11
13. Limitations and future work 12
Limitations 12
Future work 12

1. The project in one minute

Routers decide which path each packet takes through a network. Classic routing protocols such as **OSPF** and **RIP** choose the path with the smallest fixed number, either the configured delay or the number of hops. They do not notice when a link starts dropping packets, gets congested or keeps failing and coming back. They only react when a link completely stops working.

This project builds an **Intelligent Routing Protocol** that measures each link continuously (delay, packet loss, free bandwidth and stability), combines those measurements into one cost per link, and then uses **Dijkstra's algorithm** to choose the cheapest path. When the network changes, the costs change and the route is recalculated automatically. Extra rules stop it from switching paths back and forth too often.

It is tested in a simulator against three standard approaches over nine fault scenarios, each repeated with 10 random seeds. It is **statistically better than OSPF-like routing in every fault scenario**, and behaves identically to OSPF on a healthy network.

> **One-line pitch:** "OSPF picks the shortest path. My protocol picks the path that is actually working best right now, and it does this without becoming unstable."

2. Background terms you need

You will use these words when presenting. Each one is explained in plain language.

Term	What it means
Router / node	A device that forwards packets. In the diagrams the routers are the letters A, B, C and so on.
Link	A connection between two routers, such as B-E.
Path / route	The sequence of routers a packet travels through, such as A→B→E→H.
Latency (delay)	How long a packet takes to cross a link, in milliseconds (ms).
Packet loss	The fraction of packets that never arrive. 15% loss means 15 of every 100 are lost.
Bandwidth / capacity	How much data a link can carry per second, in Mb/s.
Congestion	When more traffic is sent than a link can carry, so queues build up, delay rises and packets are dropped.
Queueing delay	Extra waiting time inside a router when the link is busy. It rises sharply as the link gets close to full.
Link flapping	A link that keeps going down and coming back up.
OSPF	Open Shortest Path First. A widely used link-state protocol. It uses a fixed cost per link and runs Dijkstra.
RIP	Routing Information Protocol. Counts hops only: every link costs 1.
Dijkstra's algorithm	The standard algorithm for finding the lowest-cost path in a graph when costs are not negative.
Probe	A small test message sent over a link to measure delay and loss (like a ping).

Term	What it means
Dead timer	A link is declared down after a set number of probes in a row go missing. Here it is 3 probes.
EWMA	Exponentially Weighted Moving Average. A running average that gives more weight to recent readings, so one noisy sample does not cause a big reaction.
Hysteresis	Requiring a clear improvement (here 15%) before switching, so small changes do not cause route changes.
Hold-down	A minimum waiting time (here 3 s) between route changes.
Seed	A starting number for the random generator. Running with 10 seeds means 10 different random versions of each experiment.
Welch's t-test / p-value	A statistical test that checks whether a difference between two sets of results is real or just luck. $p < 0.05$ means it is very unlikely to be luck.

3. The problem it solves

The README uses a simple four-router example:

```
A → B → D     latency = 20 ms,  loss = 15%
A → C → D     latency = 35 ms,  loss = 1%
```

Shortest-path routing chooses **A→B→D** because 20 ms is less than 35 ms. But that path loses 15 of every 100 packets. Lost packets have to be resent, which makes applications stall, video freeze and downloads slow down. The extra 15 ms on A→C→D is much less harmful than 15% loss, so **A→C→D is the better path**. This project chooses A→C→D.

More generally, classic protocols have three weaknesses that this project addresses:

- **They ignore loss, congestion and bandwidth.** A link can be almost unusable and still look like the best choice.
- **They react slowly to failures.** They wait for the dead timer (3 missed probes) before moving traffic.
- **They cannot cope well with flapping links.** Each time the link comes back, traffic moves back onto it, and each time it fails, traffic is lost again.

4. How the protocol works

The system has five stages. Data flows from the network, through measurement and scoring, to a route decision.

Stage	What happens	File
1. Network (simulator)	Holds the true state of every router and link. Runs in 100 ms steps. Faults such as a delay spike or a failure are applied here.	`simulator.py`, `topology.py`

Stage	What happens	File
2. Link Monitor	Every 0.5 s it probes each link and measures delay, loss and load. Readings are noisy, so it smooths them with an EWMA. It also counts missed probes and remembers recent failures.	`monitor.py`
3. Cost function	Turns each link's measurements into one number (the cost). Different policies use different cost functions.	`policies.py`
4. Dijkstra	Finds the lowest-cost path from source to destination using those costs.	`engine.py`
5. Stability checks	Decides whether to actually switch to the new path (hysteresis, hold-down). Records the reason for every change.	`engine.py`

> **Important design point:** the routing engine never reads the true network state. It only sees what the monitor measures, so its information is noisy and slightly delayed, as it would be on a real network. All four policies use the same monitor, so the comparison is fair. The only thing that differs between them is the routing logic.

5. The cost formula, explained

Each link gets a cost in "effective milliseconds". Everything is converted to the same unit so the parts can be added together:

```
cost = 1   × (latency + queueing delay)
     + 300 × −ln(1 − loss)
     + 1   × 100 / free bandwidth (Mb/s)
     + 25  × flap score
```

5.1 Each part in plain words

- **Delay part.** The measured latency plus the queueing delay the link would have **if this flow's traffic were added to it**. A link that is almost full gets a large queueing delay, so it looks expensive before any traffic is sent onto it.
- **Loss part.** Loss is weighted heavily (×300) because lost packets hurt users much more than a few milliseconds of delay. The form −ln(1 − p) is a maths trick, explained below. If a link would overflow, the expected overflow loss is added too.
- **Bandwidth part.** 100 divided by the free capacity. A link with lots of spare capacity costs about 1. A link with almost no spare capacity costs a lot.
- **Stability part.** Each time a link goes down, its flap score goes up by 1. The score halves every 20 seconds. A link that keeps failing therefore stays expensive for a while, and traffic stays away from it.

5.2 Why −ln(1 − p) for loss?

Dijkstra adds costs along a path. But loss does not add up: if two links each deliver 90% of packets, the path delivers 0.9 × 0.9 = 81%, which is a product. Taking the logarithm turns a product into a sum: −ln(0.9 × 0.9) = −ln(0.9) + (−ln(0.9)). So using −ln(1 − p) as the cost means that **adding costs**

along a path is exactly the same as multiplying delivery rates, and Dijkstra minimises end-to-end loss exactly, not approximately.

5.3 Worked example (real output from the demo)

Running `python -m irp demo` on the four-router example prints the cost of each link:

Link	Delay part	Loss part	Bandwidth part	Total
A-B	10.8	0.3	1.0	12.1
B-D (15% loss)	10.8	48.8	1.0	60.5
A-C	18.2	1.5	1.0	20.8
C-D	18.2	1.5	1.0	20.8

Path A→B→D costs 12.1 + 60.5 = **72.6**. Path A→C→D costs about 20.8 + 20.8 = **41.5**. The protocol therefore chooses **A→C→D**. OSPF only looks at delay (20 vs 35), so it chooses A→B→D.

6. Keeping it stable

A protocol that reacts to every small change would keep switching paths. This is called **route oscillation** or route flapping, and it is harmful because packets get reordered and lost during each switch. The project uses five rules to prevent it:

Rule	How it works	Why it matters
Hysteresis (15%)	Only switch if the new path is at least 15% cheaper than the current one.	Small, noisy differences do not cause switches.
Hold-down (3 s)	At most one cost-based change every 3 seconds. Failures skip this rule.	Limits how often routes can change, but failures are still handled immediately.
Flap damping (20 s half-life)	Each failure adds a penalty that fades over time.	Traffic stays away from a link that keeps failing.
Early failure detection	A missed probe counts as packet loss.	Traffic moves off a dying link after 1 missed probe instead of waiting for 3.
No self-avoidance	A flow's own traffic is subtracted from a link's measured load.	Without this, a flow would see the congestion it caused itself and run away from it, then come back, and so on forever.

Every route change also records **why** it happened, for example: `loss on B-E degraded: loss ≈30.2%, util 40% (73% cheaper)`. This makes the protocol easy to explain and debug.

7. The code, file by file

The core protocol uses only the Python standard library. pandas, scipy, matplotlib, plotly and streamlit are used only for the evaluation, graphs and dashboard.

File	What it contains
irp/topology.py	The Network, Link and LinkState classes, plus the queueing-delay and overflow-loss formulas.
irp/monitor.py	LinkMonitor: sends probes, smooths readings with EWMA, runs the dead timer and tracks flaps.
irp/policies.py	The four routing policies: adaptive (ours), OSPF-like, latency-only and hop count. Each is just a different link cost function.
irp/engine.py	RoutingEngine: Dijkstra, hysteresis, hold-down and the reason for each route change.
irp/simulator.py	Discrete-time simulator in 100 ms steps, and the fault events (fail, restore, degrade).
irp/scenarios.py	The test topologies (8-router mesh and the 4-router diamond) and the 9 experiments.
irp/metrics.py	Computes delivery ratio, throughput, latency, recovery time and time degraded.
irp/evaluation.py	Runs every scenario with 10 seeds, computes 95% confidence intervals and t-tests, draws the graphs and writes `results/REPORT.md`.
irp/__main__.py	The command-line interface (demo, scenarios, run, evaluate, dashboard).
dashboard/app.py	Interactive Streamlit dashboard.
tests/test_irp.py	29 automated tests (pytest).
results/	Output of the evaluation: CSV files, graphs and the full report.

8. How it was tested

8.1 The test network

Most experiments use an 8-router mesh with 13 links:

```
        B ------ E
      / |        | \
A - C ------ F - H      A→B→E→H  20 ms (shortest)
      \ |        | /    A→C→F→H  22 ms
        D ------ G      A→D→G→H  45 ms, but 1 Gb/s
```

The flow being tested, **f1**, goes from A to H at 40 Mb/s. A second flow, **f2**, goes from C to E at 30 Mb/s as competing traffic. Each run lasts 60 seconds. Faults start at 15 s and clear at 40 s, so each fault lasts 25 seconds.

8.2 The nine experiments

Scenario	What happens
Steady	No faults. Checks that the protocol costs nothing on a healthy network.
Latency spike	Delay on B-E rises from 10 ms to 150 ms.
Bandwidth drop	Capacity of E-H falls from 100 Mb/s to 15 Mb/s.

Scenario	What happens
Packet loss increase	Random loss on B-E rises from 0.1% to 12%.
Link failure	B-E goes down, then comes back at 40 s.
Congestion	Background traffic on B-E ramps from 50 to 95 Mb/s.
Flapping link	E-H fails for 2 s every 4 s.
Random faults (chaos)	Seven random faults per run.
Lossy shortcut	The A→B→D vs A→C→D example; B-D is repaired at 30 s.

8.3 What it is compared against

- **OSPF-like static routing** (main baseline): uses the configured latency as the cost and only reacts when a link goes down or comes back.
- **Latency-only dynamic routing:** reacts to measured delay and queueing, with the same stability rules. This shows what you get from delay alone.
- **Hop count (RIP-like):** every link costs 1.

8.4 What is measured

- **Delivery ratio:** packets delivered ÷ packets sent.
- **Throughput ratio:** data actually delivered ÷ data the flows wanted to send.
- **Mean and p95 latency:** average delay, and the delay that 95% of packets beat.
- **Recovery time:** seconds from the fault until f1 is back to at least 90% of its normal throughput.
- **Time degraded:** seconds during the fault when f1 was below 90% of its normal throughput.
- **Route changes:** how many times routes switched (fewer is more stable).

Each experiment is run with **10 different random seeds**. Results are reported as mean ± 95% confidence interval, and **Welch's t-test** checks whether the difference from OSPF is significant ($p < 0.05$).

9. Results

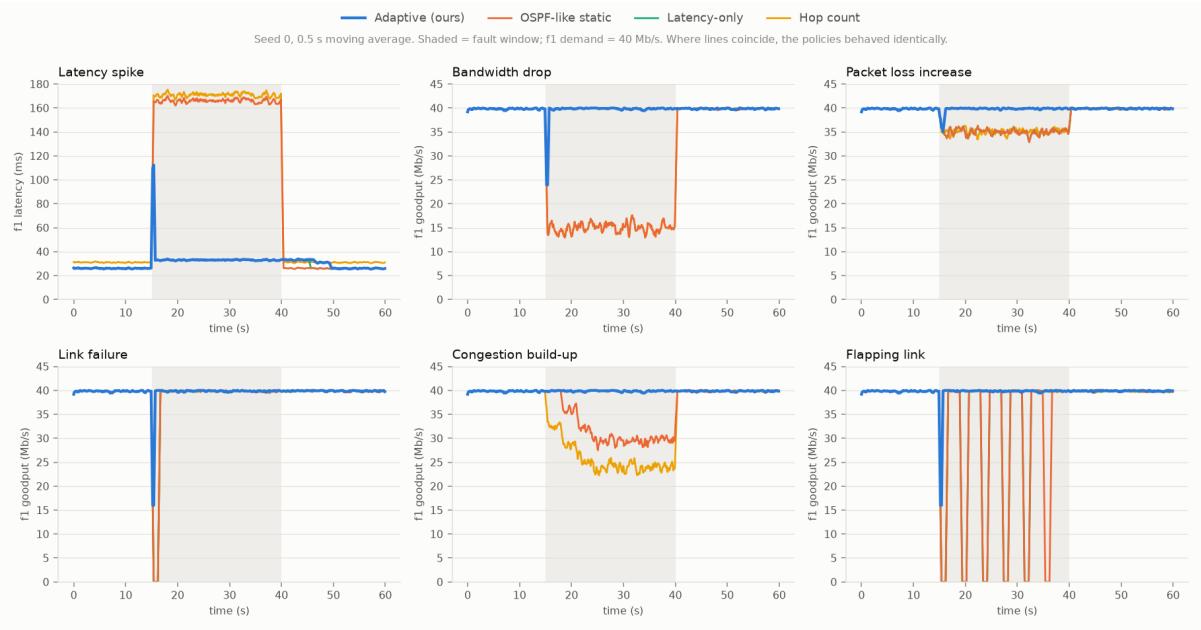
The table compares the adaptive protocol with OSPF-like routing. Numbers are averages over 10 seeds.

Scenario	Delivery ratio	Time degraded	Mean latency
Steady state	0.997 vs 0.997 (same)	–	20.3 vs 20.3 ms (same)
Latency spike	same	same	**24.0 vs 49.4 ms**
Bandwidth drop	**0.996 vs 0.867**	**0.5 vs 24.9 s**	21.8 vs 41.6 ms
Packet loss increase	**0.997 vs 0.973**	**0.1 vs 23.6 s**	23.6 vs 20.1 ms (worse)
Link failure	0.996 vs 0.988	**0.7 vs 2.0 s**	25.0 vs 22.8 ms (worse)
Congestion	**0.997 vs 0.956**	**0.0 vs 19.5 s**	23.9 vs 57.2 ms

Scenario	Delivery ratio	Time degraded	Mean latency
Flapping link	**0.996 vs 0.938**	**0.7 vs 12.0 s**	21.6 vs 20.6 ms (worse)
Random faults	0.996 vs 0.966	–	23.4 vs 30.9 ms
Lossy shortcut	**0.994 vs 0.924**	–	29.5 vs 21.5 ms (worse)

9.1 What the results mean

- **Reliability:** in every fault scenario except the latency spike (where every policy still delivers all its traffic), the adaptive protocol delivers more traffic, and the difference is statistically significant. On a healthy network it is identical to OSPF.
- **Recovery:** after a link failure it recovers in about 0.13 s versus 1.15 s for OSPF, roughly **9× faster**, because it moves after one missed probe instead of three.
- **Flapping:** OSPF changes route 12 times and loses about 6% of traffic. The adaptive protocol changes route only 2 times, because flap damping keeps it on the stable path.
- **The trade-off:** in four scenarios it has a few ms more latency. This is deliberate: it uses a slightly longer but clean path instead of a short lossy one, and does not rush back to a link that just failed. The weights control this balance.
- **Why multiple metrics matter:** the latency-only baseline does as well as ours on faults that show up as delay (latency spike, bandwidth drop, congestion). It fails on packet loss, flapping and the lossy shortcut, which do not raise delay. That is the main argument for combining several metrics.



How the test flow behaved over time. Blue is the adaptive protocol and orange is OSPF-like. The grey band is the fault window.

10. How to run it

From inside the project folder:

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

python -m irp demo                            # the A-B-D example and a link failure
python -m irp scenarios                       # list all experiments
python -m irp run loss_increase -p adaptive  # one experiment with its route-change
log
python -m irp evaluate --seeds 10             # full evaluation into results/
python -m irp dashboard                       # dashboard at http://localhost:8501
pytest -q                                     # run the 29 tests
```

A virtual environment (`.venv`) already exists in the project folder, so you can usually skip the first two lines and run `source .venv/bin/activate` instead.

The **dashboard** is the best thing to show live. In the sidebar you can choose a scenario and a policy, pick a baseline to compare against, change the cost weights, and inject your own faults on any link. It then shows the topology with the chosen route, link health over time, the route-change log and the performance numbers.

11. How to pitch it (about 5 minutes)

1. **The problem (slide 2).** “Routing protocols like OSPF pick the shortest path using a fixed number. They don't notice if that path is losing packets or is congested. In this example OSPF picks a path that loses 15% of packets.”
2. **The idea (slide 3).** “My protocol measures four things on every link (delay, loss, free bandwidth and stability), combines them into one cost, and runs Dijkstra on that cost.”
3. **How it works (slides 4–6).** Walk through the five stages. Mention that the engine only sees noisy measurements, as in real life. Then explain that stability rules stop it from flip-flopping.
4. **Testing (slide 7).** “I tested it in a simulator on nine kinds of faults, against OSPF, latency-only and hop-count routing, with 10 random seeds each and a t-test.”
5. **Results (slides 8–10).** Give the four headline numbers: 9× faster recovery, 99.6% vs 86.7% delivery when bandwidth drops, 0 s vs 19.5 s degraded under congestion, and 2 vs 12 route changes on a flapping link.
6. **Honesty (slide 12).** Mention the small latency trade-off and the limitations. Teachers usually respect this.
7. **Close (slide 13).** “Same as OSPF when the network is healthy, far more reliable when it is not.” If there is time, run the dashboard.

12. Questions your teacher may ask

Q: Why not just use OSPF with different link costs?

A: OSPF costs are set by an administrator and do not change with live conditions. OSPF only recalculates when a link goes up or down. This protocol updates costs continuously from measurements, so it reacts to loss, congestion and bandwidth changes that OSPF never sees.

Q: Why is the loss weight 300?

A: It sets how much delay we would accept to avoid loss. With 300, 1% loss costs about 3 ms, and 15% loss costs about 49 ms. The weights are hand-tuned and can be changed in the dashboard. Learning them automatically is listed as future work.

Q: Why −ln(1 − p) and not just p?

A: Because delivery probabilities multiply along a path, and Dijkstra adds. The log turns multiplication into addition, so minimising the sum of −ln(1 − p) is exactly the same as maximising end-to-end delivery.

Q: Won't an adaptive protocol keep switching routes?

A: That is the main risk, and five mechanisms handle it: 15% hysteresis, a 3 s hold-down, flap damping, smoothing measurements with an EWMA, and not counting a flow's own traffic. In the flapping test it made 2 route changes compared with OSPF's 12.

Q: How does it detect failures faster than OSPF?

A: Both use the same 3-missed-probe dead timer. But in this protocol a missed probe is also counted as packet loss, so the loss part of the cost rises immediately and traffic moves after one missed probe, before the link is officially declared down.

Q: Why is latency worse in some scenarios?

A: Because it deliberately chooses a slightly longer path that does not lose packets, and it does not rush back to a link that recently failed. Avoiding 2–7% packet loss is worth a few milliseconds. The balance can be changed with the weights.

Q: Is this a real protocol running on routers?

A: Not yet. It is a simulation. Routing decisions use a shared view of all link states, and the flooding of link-state messages between routers is not modelled. Making it a distributed protocol and testing it in Mininet or ns-3 are the next steps.

Q: How do you know the results are not just luck?

A: Every experiment was run with 10 different random seeds, results are given with 95% confidence intervals, and Welch's t-test was used. Improvements were significant ($p < 0.05$) in every fault scenario.

Q: What is the time complexity?

A: Each route calculation is a Dijkstra run with a binary heap: $O((V + E) \log V)$ per flow per update, where V is routers and E is links. The cost of each link is $O(1)$ to compute.

Q: What does EWMA do here?
A: It smooths noisy probe readings: new estimate = 0.7 × old estimate + 0.3 × new sample. One unusual reading therefore cannot trigger a route change on its own.

13. Limitations and future work

Limitations

- It is a flow-level, discrete-time simulator, not a packet-level one, so TCP behaviour (such as slowing down after loss) is not modelled. Queueing uses an M/M/1 approximation.
- Routes are calculated per flow using global knowledge of all links. Distributed message flooding and convergence between routers are not modelled, apart from probe and detection delay.
- The weights were tuned by hand.

Future work

- Learn the weights automatically, for example with Bayesian optimisation over the scenario suite.
- Test with real packets in an emulator such as Mininet or ns-3.
- Turn it into a distributed protocol where routers exchange their measurements.